Product Owner Role Guide

Version: 1.0
Date: 2026-03-03
Audience: Trainees (Product Owner role)
Purpose: Clear expectations, responsibilities, and practical guidance

---

- **Product Owner Role Guide**
  - Role Overview
    - What is a Product Owner?
    - Core Responsibility
  - Key Responsibilities
    - 1. Product Backlog Management
    - 2. Vision & Value
    - 3. Stakeholder Management
    - 4. Sprint Participation
    - 5. Acceptance & Quality (Using Definition of Done)
  - Concrete Tasks
    - During Sprint 0 (Preparation Week)
    - During Each Sprint (Ongoing)
    - Between Sprints
  - How to Write Good User Stories
    - User Story Format
    - Example: Good User Story
    - Example: Bad User Story (and Why)
    - INVEST Criteria Checklist
  - Good Practice Scenarios
    - Scenario 1: Effective Sprint Planning
    - Scenario 2: Handling Mid-Sprint Clarification
    - Scenario 3: Sprint Review with Stakeholder Feedback
  - Anti-Patterns (What Goes Wrong)
    - Anti-Pattern 1: Absent Product Owner
    - Anti-Pattern 2: Requirements Dictator
    - Anti-Pattern 3: Vague Storyteller
    - Anti-Pattern 4: Scope Creep Champion
  - Common Challenges & Solutions
    - Challenge 1: "I don't know the answer to technical questions"
    - Challenge 2: "Stakeholder wants everything NOW"
    - Challenge 3: "Team keeps saying stories aren't ready"
    - Challenge 4: "I feel like I'm not doing enough"
  - Quick Reference Checklists
    - Sprint 0 Checklist
    - Sprint Planning Checklist
    - Daily During Sprint
    - Mid-Sprint Refinement Checklist
    - Sprint Review Checklist
    - Sprint Retrospective Checklist
  - Tips for Success
    - 1. Be Decisive
    - 2. Be Available

3. Be Clear
4. Prioritize Ruthlessly
5. Say No
6. Collaborate, Don't Dictate
Getting Help
When to Ask Staff for Help
When to Work with Scrum Master
When to Collaborate with Team
Remember

Role Overview

What is a Product Owner?

The Product Owner (PO) is **responsible for maximizing the value of the product** and the work of the Development Team.

You are NOT:

- ❌ A project manager telling people what to do
- ❌ A requirements gatherer who just passes messages
- ❌ Someone who writes everything down and disappears

You ARE:

- ✅ The voice of the stakeholders and users
- ✅ The decision-maker on what gets built and when
- ✅ The guardian of product value and vision
- ✅ The person who says "yes" or "no" to features

Core Responsibility

One sentence: You decide **what** gets built and **why**, the team decides **how**.

Key Responsibilities

1. Product Backlog Management

You own the Product Backlog. This means:

Creating:

- Write user stories with acceptance criteria
- Ensure stories are clear, testable, and valuable
- Include GDPR requirements where relevant

Prioritizing:

- Order the backlog by value/urgency
- Make trade-off decisions (what's more important?)
- Re-prioritize based on changing needs

Refining:

- Ensure top stories are ready for Sprint Planning
- Split large stories into smaller ones
- Clarify requirements with the team

Maintaining:

- Remove outdated stories
- Update priorities as needed
- Keep backlog organized and visible

2. Vision & Value

You communicate WHY we're building this:

- Share the Product Vision with the team
- Explain the value of each feature
- Help team understand user needs
- Make decisions that maximize value

3. Stakeholder Management

You are the bridge between stakeholders and team:

- Gather requirements from stakeholders (staff, DPO for GDPR)
- Communicate progress and decisions
- Manage expectations about what's possible
- Shield team from conflicting demands

4. Sprint Participation

You are actively involved in all Sprint events:

- **Sprint Planning:** Present and clarify user stories
- **Daily Standup:** Available for questions (optional attendance)
- **Sprint Review:** Present the Increment to stakeholders
- **Sprint Retrospective:** Participate and improve collaboration

5. Acceptance & Quality (Using Definition of Done)

You decide if work is "Done":

Two levels of Done:

1. Story-level: Acceptance Criteria

- Review completed work against **story-specific** acceptance criteria
- Example: "Email sent within 5 seconds" → Test this specific requirement

2. Product-level: Definition of Done

- Review completed work against **general quality standards**
- Definition of Done = checklist that applies to EVERY story

Definition of Done (EAP Template - Your team customizes this in Sprint 0):

Every user story is only "Done" when:

Code:

- ☐ Code is readable and reviewed by at least one team member
- ☐ No critical warnings or errors
- ☐ Code follows agreed coding standards

Testing:

- ☐ Unit tests are added or updated where relevant
- ☐ Tests pass in CI
- ☐ Test coverage is adequate for the changes made

DevOps:

- ☐ Build succeeds
- ☐ Relevant automated tests are executed as part of the pipeline
- ☐ Deployment works in the target environment (if applicable)

Quality & Resilience:

- ☐ The application behaves predictably under failure conditions
- ☐ Errors are logged clearly
- ☐ Recovery or rollback is possible

Documentation:

- ☐ Changes are documented
- ☐ Setup instructions are updated (if applicable)
- ☐ API documentation is updated (if applicable)

Architecture:

- ☐ Architectural impact is documented (if applicable)

- ☐ If significant architectural decisions were made:
  - Architecture Decision Record (ADR) is written
  - ADR is reviewed and accepted by the team
  - Architecture diagrams are updated (if needed)

Privacy & Data Protection:

- ☐ Personal data handling is reviewed (if applicable)
- ☐ No sensitive data exposed in logs or error messages
- ☐ Data retention implications are considered
- ☐ Access controls prevent unauthorized data access
- ☐ Privacy by design principles are followed

Agreement:

- ☐ The team agrees this work is "Done"
- ☐ **Product Owner has reviewed and accepted the work** (that's you!)
- ☐ Done means ready to show in Sprint Demo and ready to be used by others

Note: Not every story requires all checkpoints. For example:

- Small bug fixes may not need architecture documentation
- UI-only changes may not affect API documentation
- Internal refactoring may not need updated setup instructions

Use professional judgment as a team. When in doubt, document.

Your team creates their final Definition of Done during Sprint 0, based on this template.

Your role with Definition of Done:

Before Sprint:

- Help team customize DoD for your product (if needed)
- Ensure DoD is visible and understood

During Sprint:

- When team says "Story X is done," check DoD
- If anything is missing: "Not done yet, please complete [missing item]"
- Don't accept "done" until ALL DoD items are checked

Sprint Review:

- Only demo work that meets DoD
- Don't show "90% done" work - it's not done

Why this matters:

- DoD prevents technical debt (cutting corners)
- Ensures consistent quality

- "Done" means actually shippable, not "mostly done"

Common mistake:

- ❌ Accepting work that "mostly works" because sprint is ending
- ✅ Better: Accept only truly Done work, move incomplete work to next sprint

Note: Definition of Done is typically created by the team during Sprint 0, with your input on quality expectations.

Concrete Tasks

During Sprint 0 (Preparation Week)

Your ONLY job this week: Build the Product Backlog

Day 1-2:

- [ ] Read Product Vision thoroughly
- [ ] Receive GDPR requirements from staff/DPO
- [ ] Understand privacy by design principles
- [ ] Make notes on required features

Day 2-5:

- [ ] Write 15-20 user stories with acceptance criteria
- [ ] Include GDPR-related stories (data export, anonymization, consent, etc.)
- [ ] Ensure each story has clear value proposition
- [ ] Prepare for team refinement sessions

Day 3-5:

- [ ] Facilitate daily refinement sessions with team (30-60 min)
- [ ] Answer team questions about requirements
- [ ] Adjust stories based on technical input
- [ ] Get team feedback on clarity and testability

Day 4-5:

- [ ] Facilitate Planning Poker estimation sessions
- [ ] Ensure top 10 stories are estimated **by the team** (you facilitate, they estimate)
- [ ] Prioritize backlog based on value and dependencies
- [ ] Prepare Sprint 1 Planning agenda

Important: You facilitate estimation, the **team provides the estimates**. You never estimate story points yourself - this is the team's responsibility based on their technical understanding.

End of Sprint 0:

- [ ] Product Backlog has 15-20 ready stories

- ☐ Top 10 stories are estimated and detailed
- ☐ Team understands the product vision
- ☐ **Team has customized Definition of Ready** (based on template)
- ☐ **Team has customized Definition of Done** (based on template)
- ☐ Sprint 1 Planning agenda is ready

Success criteria: Team can walk into Sprint 1 Planning and immediately select stories without confusion.

Note: Definition of Ready and Definition of Done are created collaboratively during Sprint 0. You provide input on quality expectations and clarity requirements, but the team owns the final version.

During Each Sprint (Ongoing)

Beginning of Sprint (Sprint Planning):

- ☐ Present top priority user stories
- ☐ Explain the value/goal of each story
- ☐ Answer clarifying questions
- ☐ Help team commit to achievable Sprint Goal
- ☐ Ensure Sprint Goal aligns with product vision

During Sprint (Daily/As Needed):

- ☐ Be available for questions (within 24 hours response)
- ☐ Clarify acceptance criteria when asked
- ☐ Make decisions on scope/priorities quickly
- ☐ Attend Daily Standup (optional but recommended)
- ☐ Review work in progress if team requests
- ☐ **When team says "story is done," verify it meets Definition of Done before accepting**

Mid-Sprint (Refinement Session - once per sprint):

- ☐ Facilitate backlog refinement (1-2 hours)
- ☐ Present upcoming stories for next sprint
- ☐ Answer questions, adjust stories based on feedback
- ☐ Ensure next sprint's stories are "Ready"

End of Sprint (Sprint Review):

- ☐ Prepare demo script (what to show, what value it delivers)
- ☐ Present completed work to stakeholders
- ☐ Gather feedback from stakeholders
- ☐ Adjust backlog based on feedback

End of Sprint (Sprint Retrospective):

- ☐ Participate in team retrospective

- [ ] Listen to team concerns about collaboration
- [ ] Commit to improvements in your role

Between Sprints

- [ ] Refine backlog (add/remove/update stories)
- [ ] Engage with stakeholders (gather feedback)
- [ ] Prepare for next Sprint Planning
- [ ] Write new user stories if needed

How to Write Good User Stories

User Story Format

```
As a [type of user]
I want [goal/desire]
So that [benefit/value]

Acceptance Criteria:
- [ ] Specific testable condition 1
- [ ] Specific testable condition 2
- [ ] Specific testable condition 3

GDPR Considerations (if applicable):
- Personal data involved: [specify]
- Legal basis: [consent/contract/legitimate interest]
- Privacy by design: [how is privacy protected]
```

Example: Good User Story

```
USER STORY: Employee Self-Service Request Creation

As an employee
I want to submit a leave request through a web form
So that I can request time off without sending emails

Acceptance Criteria:
- [ ] Form includes: start date, end date, type of leave (vacation/sick/other)
- [ ] Form validates dates (end date must be after start date)
- [ ] Form shows confirmation message after submission
- [ ] Employee receives email confirmation with request details
- [ ] Request appears in "My Requests" list with status "Pending"

GDPR Considerations:
- Personal data: Employee name, email, leave dates
- Legal basis: Employment contract (necessary for employment)
- Privacy by design: Only employee and approver can see request details
```

Example: Bad User Story (and Why)

```
❌ BAD:
"The system should have a request form"

Why it's bad:
- Who is the user? (employee? manager? admin?)
- What kind of request?
- What value does it provide?
- No acceptance criteria (how do we know it's done?)
- Not testable
```

```
✅ BETTER:
"As an employee, I want to submit a leave request through a form
so that I can request time off efficiently."

[Then add specific acceptance criteria]
```

INVEST Criteria Checklist

Good user stories are **INVEST**:

- **I**ndependent: Can be built without depending on other stories
- **N**egotiable: Details can be discussed, not a contract
- **V**aluable: Delivers value to users
- **E**stimable: Team can estimate effort
- **S**mall: Can be completed in one sprint
- **T**estable: Clear acceptance criteria

Good Practice Scenarios

Scenario 1: Effective Sprint Planning

Context: Sprint 1 Planning, team is ready to select stories.

What PO Does:

Before the meeting:

- Reviews top 10 stories in backlog
- Ensures all have acceptance criteria
- Prepares to explain value of each story

During Sprint Planning:

> **PO:** "Our Sprint Goal is to enable employees to submit basic leave requests. I've prioritized three stories for this sprint:

> **Story 1:** Employee can create leave request
> **Value:** This is the core feature employees need most.
> **Team's estimate:** 8 points
>
> **Story 2:** Employee can view their request history
> **Value:** Employees need to track what they've requested.
> **Team's estimate:** 5 points
>
> **Story 3:** Request validation (start/end dates)
> **Value:** Prevents invalid requests, reduces manual checking.
> **Team's estimate:** 3 points
>
> Total: 16 points. In Sprint 0, the team estimated your velocity at 15-20 points per sprint, so this seems achievable.
>
> **Question for team:** Can we commit to these three stories? Any concerns?"

Team: "Story 1 seems large. Can we split it?"

PO: "Good point. Let's split it:

- Story 1A: Basic form (name, dates, type)
- Story 1B: Email confirmation

Team, can you estimate these two pieces?"

Team: "[Brief Planning Poker] 1A is 5 points, 1B is 3 points."

PO: "Perfect, that gives us more flexibility. So our sprint is: 1A (5), 1B (3), Story 2 (5), Story 3 (3) = 16 points total. Team, can we commit?"

Team: "Yes, we commit to this scope."

Result:

- ✅ Clear Sprint Goal
- ✅ Stories prioritized by value
- ✅ Team involved in scope decision
- ✅ Realistic commitment
- ✅ Flexibility to adapt

Scenario 2: Handling Mid-Sprint Clarification

Context: Day 3 of sprint, developer has question about acceptance criteria.

Developer: "Story says 'employee receives email confirmation.' Should this be immediate or can it be queued?"

❌ **BAD PO Response:**

> "I don't know, what do you think?"

Why it's bad: PO abdicated decision-making. This is YOUR decision.

✅ **GOOD PO Response:**

> "Good question. Immediate is better for user experience - they should see confirmation right away. Queue it only if technical constraints require it. What's the impact?"

Developer: "Immediate is doable, just wanted to confirm."

PO: "Perfect, let's go with immediate. I'll update the acceptance criteria to be explicit: 'Email sent within 5 seconds of submission.'"

Why it's good:

- PO made decision based on user value
- Asked for technical input
- Updated story for clarity
- Quick response (same day)

Scenario 3: Sprint Review with Stakeholder Feedback

Context: End of Sprint 1, presenting to staff stakeholders.

PO presents demo:

> **PO:** "In Sprint 1, we achieved our goal: employees can now submit leave requests.
>
> **Demo:** [Shows form, submission, confirmation, request list]
>
> **Value delivered:** Employees no longer need to send email requests. The form validates dates automatically, reducing errors.
>
> **Next sprint:** We'll add the approval workflow so managers can approve/reject requests."

Stakeholder: "This looks good, but I notice there's no field for 'reason for leave.' We need that for HR reporting."

❌ **BAD PO Response:**

> "Oh, the team didn't build that. We can add it next sprint."

Why it's bad: Sounds like PO didn't know this was needed. Implies poor requirements gathering.

✅ **GOOD PO Response:**

> "Good point. I didn't include that in this sprint because I prioritized getting the basic flow working first. Adding a 'reason' field is definitely valuable for HR reporting. I'll add that story to the backlog and we can prioritize it for Sprint 2 or 3. Should this be required or optional?"

Stakeholder: "Optional is fine."

PO: "Perfect, I'll write that story this week. [Takes notes]"

Why it's good:

- Acknowledged the feedback professionally
- Explained prioritization decision
- Committed to action (backlog story)
- Asked clarifying question (required/optional)
- Documented it immediately

Anti-Patterns (What Goes Wrong)

Anti-Pattern 1: Absent Product Owner

What it looks like:

Week 1: PO writes user stories.
Week 2-6: PO disappears, team never sees them.
Sprint Planning: Team asks questions, PO says "I don't remember, let me check" repeatedly.
Sprint Review: PO is unprepared, demo is chaotic.

Why it's bad:

- Team blocked waiting for decisions
- Stories aren't refined, team guesses at requirements
- No prioritization, team picks randomly
- No value focus, team builds wrong things
- Sprint Reviews are embarrassing

Symptoms:

- Team says: "We can't progress, waiting for PO"
- Stories are vague or contradictory
- Team builds features PO didn't want
- Stakeholders frustrated with slow progress

How to fix it:

Immediate:

- **Set availability hours:** "I'm available on Teams daily 10-12 and 14-16"
- **Attend Daily Standup:** Even just listening helps you stay connected
- **Respond within 24 hours:** To all team questions
- **Schedule weekly refinement:** 1-hour session to review upcoming stories

Long-term:

- **Make it a habit:** Block calendar time for PO work (2-3 hours/day)
- **Be proactive:** Ask team "Any questions?" rather than waiting
- **Visible backlog:** Keep Jira updated so team always knows priorities

Anti-Pattern 2: Requirements Dictator

What it looks like:

Sprint Planning:

> **PO:** "You MUST build these 5 stories this sprint. No discussion."
>
> **Developer:** "That's 30 points, we only did 18 last sprint—"

> **PO:** "Then work harder. These are all critical."
>
> **Scrum Master:** "Maybe we should—"
>
> **PO:** "I'm the PO, I decide what gets built. Just do it."

Mid-Sprint:

> **Developer:** "This acceptance criteria is impossible to test. Can we adjust—"
>
> **PO:** "No changes. Build it exactly as written."

Sprint Review:

> **PO to team:** "You failed to deliver everything I asked for. This is unacceptable."

Why it's bad:

- Team morale collapses
- Collaboration breaks down
- Team stops asking questions (fear of rejection)
- Unrealistic commitments lead to poor quality
- PO seen as antagonist, not partner

Symptoms:

- Team is quiet in Planning (no discussion)
- Stories regularly incomplete at Sprint end
- Quality issues increase
- Team-PO relationship is tense
- Retrospectives filled with complaints about PO

How to fix it:

Immediate:

- **Apologize:** "I've been too demanding. Let's collaborate better."
- **Listen to capacity:** "Team, what's realistic for this sprint?"
- **Be flexible:** "If we can only do 3 stories, which 3 deliver most value?"

Mindset shift:

- **You decide WHAT & WHY, team decides HOW & HOW MUCH**
- **Team velocity is reality, not negotiable**
- **Collaboration > Dictation**
- **Team's technical input is valuable**

Practices:

- **Ask, don't tell:** "Can we achieve X?" not "You must do X"
- **Prioritize ruthlessly:** "If we can only do 3 of these 5, let's pick the most valuable 3"
- **Respect team capacity:** "Last sprint was 18 points, so let's plan for 15-20"

Anti-Pattern 3: Vague Storyteller

What it looks like:

User Story Example:

```
As a user
I want the system to be better
So that it's easier to use

Acceptance Criteria:
- Make it good
- Users should like it
```

Sprint Planning:

> **Developer:** "What does 'better' mean specifically?"
>
> **PO:** "You know, just... better. More user-friendly."
>
> **Developer:** "Can you give specific examples?"
>
> **PO:** "Just use your judgment. You're the developers."

Mid-Sprint:

> **Developer:** "We built feature X, is this what you wanted?"
>
> **PO:** "Hmm, not really. I meant something different."
>
> **Developer:** "Can you clarify what you want?"
>
> **PO:** "I'll know it when I see it."

Sprint Review:

> **PO:** "This isn't what I wanted. It's not user-friendly enough."
>
> **Team:** "You didn't specify what user-friendly means!"

Why it's bad:

- Team guesses at requirements
- Rework wastes time and morale
- "Done" is undefined, constant rejection
- Team frustrated by moving goalposts
- No way to estimate vague requirements

Symptoms:

- Stories have no acceptance criteria
- Team asks many clarification questions
- Frequent rework ("that's not what I meant")
- PO rejects work but can't explain why

- Team velocity drops (constant changes)

How to fix it:

Immediate:

- **Write specific acceptance criteria:** Use checkboxes, measurable conditions
- **Use examples:** "E.g., when user enters invalid date, show error message 'End date must be after start date'"
- **Define "user-friendly":** Break it down into specific behaviors

Example - Before & After:

❌ **BEFORE (Vague):**

```
Story: Improve request form
Acceptance Criteria:
- Form should be user-friendly
- Make it look professional
```

✅ **AFTER (Specific):**

```
Story: Improve Request Form Validation

As an employee
I want immediate feedback on form errors
So that I can correct mistakes before submitting

Acceptance Criteria:
- [ ] If start date is after end date, show error: "End date must be after start date"
- [ ] If leave type is not selected, show error: "Please select leave type"
- [ ] Error messages appear immediately (< 1 second) when field loses focus
- [ ] Error messages displayed in red text above the relevant field
- [ ] Submit button disabled until all errors are resolved
- [ ] Example: Entering start date "2026-03-15" and end date "2026-03-10" triggers error

GDPR: No additional personal data collected
```

Practices:

- **Always include 3-5 specific acceptance criteria**
- **Use examples with actual data**
- **Define "good" with measurable criteria**
- **Ask team: "Is this clear enough to build?"**

Anti-Pattern 4: Scope Creep Champion

What it looks like:

Sprint Planning: Team commits to 3 stories (18 points).

Day 2 of Sprint:

> **PO:** "I just thought of something! We also need a 'cancel request' button. Can you add that?"
>
> **Team:** "That's a new story, it would add 5 points—"
>
> **PO:** "It's quick, just a button. Please add it."

Day 5 of Sprint:

> **PO:** "Can we change the email format? I want it to include the manager's name."
>
> **Developer:** "That wasn't in acceptance criteria, and we're mid-sprint—"
>
> **PO:** "It's a small change, just do it."

Day 9 (Sprint Review):

> **Stakeholder:** "Why isn't the cancel button working?"
>
> **Team:** "We didn't finish it, we ran out of time."
>
> **PO:** "But I asked for it early in the sprint!"

Why it's bad:

- Original commitment becomes impossible
- Team stress increases, quality drops
- Sprint Goal abandoned
- Velocity unpredictable
- Team stops trusting Sprint Planning

Symptoms:

- New requirements added mid-sprint frequently
- Team rarely completes Sprint Goal
- Developers working overtime
- Quality issues increase
- Retrospectives: "PO keeps changing scope"

How to fix it:

Immediate:

- **Stop adding to sprint:** "Good idea! I'll add it to backlog for next sprint."
- **Respect Sprint commitment:** Current sprint scope is frozen

When you have new idea mid-sprint:

❌ **DON'T:**

> "Can you add this now?"

✅ **DO:**

> "I have a new idea: [explain]. This is valuable, but I'll add it to the Product Backlog for Sprint N+1. Let's stay focused on our current Sprint Goal."

Exception: Only change scope if:

- **Critical bug** discovered (discuss with team and Scrum Master)
- **Regulatory requirement** emerges (rare, discuss urgency)

And only if:

- **Team agrees** they can absorb it
- **Remove equal scope** from sprint to compensate

Practices:

- **Keep a "Future Ideas" list:** Note ideas during sprint, review later
- **Protect Sprint Goal:** "Anything not supporting Sprint Goal waits for next sprint"
- **Trust the cadence:** "We have a sprint every 2 weeks, new ideas can wait"

Common Challenges & Solutions

Challenge 1: "I don't know the answer to technical questions"

Example: Developer asks "Should we use REST or GraphQL for the API?"

Solution:

- **This is NOT your decision** - this is a technical/architectural decision
- **Your response:** "That's a technical decision for the team. My requirement is: employees can submit requests via a web form. How you build the API is up to you."
- **Focus on WHAT, not HOW**

When you DO need to decide:

- "Should requests be editable after submission?" → **Product decision, you decide**
- "Should we show 10 or 50 requests per page?" → **UX decision, you decide** (or gather user feedback)

Challenge 2: "Stakeholder wants everything NOW"

Example: Staff says "We need all these 20 features by Sprint 3."

Solution:

- **Your job: Manage expectations**
- **Response:** "I understand all 20 are valuable. Team velocity is ~18 points/sprint. Over 3 sprints, that's ~54 points total. Let's prioritize the top features that deliver the most value. Which 3-5 features are absolutely critical?"
- **Show the trade-off:** "If we try to build all 20 quickly, quality suffers. If we focus on the top 10, we deliver solid value."

Then:

- Prioritize ruthlessly
- Communicate regularly what's possible

- Celebrate delivered value

Challenge 3: "Team keeps saying stories aren't ready"

Example: In Sprint Planning, team says "These stories aren't detailed enough."

Root cause: You're not using the Definition of Ready effectively.

Solution:

1. Use Definition of Ready as YOUR checklist:

Before Sprint Planning, every story in the top of your backlog should meet the team's Definition of Ready.

EAP Definition of Ready (Template - Your team customizes this in Sprint 0):

A backlog item is **Ready** when:

Clarity:

- ☐ The goal of the item is clear
- ☐ The value for the product or user is understood
- ☐ Acceptance criteria are written and understandable

Scope:

- ☐ The item is small enough to be completed within one Sprint
- ☐ Dependencies are identified
- ☐ Open questions are known and discussed

Feasibility (including DevOps):

- ☐ The team understands in general how the work can be done
- ☐ Required environments, services, or integrations are known
- ☐ Needed access, secrets, or configuration are identified

Quality & Testing:

- ☐ The team knows how the work can be tested
- ☐ Relevant test types are considered (unit tests, integration tests)

Delivery awareness:

- ☐ The team understands how this item will be delivered through the pipeline
- ☐ No known blocking issues in CI/CD or deployment exist

Agreement:

- ☐ The Product Owner agrees the item is ready (that's you!)
- ☐ **The team has estimated the story** (Planning Poker in refinement)
- ☐ The team agrees the item can be taken into a Sprint

Your responsibility: Check this BEFORE Sprint Planning. Don't bring stories that aren't "Ready."

Note: Your team creates their final Definition of Ready during Sprint 0, based on this template. You help ensure it's used consistently.

2. Schedule weekly backlog refinement:

When: Mid-sprint (e.g., Thursday of Week 1 in 2-week sprint)
Duration: 1-2 hours
Attendees: You + Team

Agenda:

- Present 5-8 stories for NEXT sprint
- Team asks clarification questions
- You adjust stories based on feedback
- Team estimates stories (Planning Poker)
- You mark stories as "Ready" when DoR is met

Result: By Sprint Planning, top stories are already understood and estimated.

3. Don't skip refinement:

❌ **Common mistake:** "We're too busy coding, skip refinement this week."

Why this fails: Next Sprint Planning, nothing is ready, Planning takes 3 hours of painful clarification.

✅ **Better:** Protect refinement time. 1-2 hours mid-sprint saves 2+ hours in Planning.

Challenge 4: "I feel like I'm not doing enough"

Common feeling: "I wrote stories in Sprint 0, now what? Team is coding, I'm just... waiting?"

Reality: You ARE doing important work:

During Sprint, you should be:

- Answering team questions (ongoing)
- Refining upcoming stories (for next sprint)
- Gathering stakeholder feedback
- Monitoring progress toward Sprint Goal
- Preparing Sprint Review
- Thinking about prioritization
- Writing new stories for future sprints

Time allocation (rough guide):

- 30% - Team collaboration (questions, refinement, ceremonies)
- 30% - Backlog management (writing, refining, prioritizing)
- 20% - Stakeholder engagement (gathering feedback, reporting)
- 20% - Sprint ceremonies (Planning, Review, Retrospective)

You are NOT idle. You are thinking, deciding, prioritizing - this is strategic work.

Quick Reference Checklists

Sprint 0 Checklist

- [ ] Read Product Vision
- [ ] Receive GDPR requirements from staff/DPO
- [ ] Write 15-20 user stories with acceptance criteria
- [ ] Include GDPR stories (consent, export, anonymization, etc.)
- [ ] Facilitate daily refinement sessions (Day 3-5)
- [ ] Run Planning Poker estimation sessions
- [ ] Prioritize backlog
- [ ] Prepare Sprint 1 Planning agenda
- [ ] Ensure team understands product vision

Sprint Planning Checklist

- [ ] Review top priority stories before meeting
- [ ] **Verify stories meet Definition of Ready** (if not, remove from Planning)
- [ ] Prepare to explain value of each story
- [ ] Present stories in priority order
- [ ] Answer clarifying questions
- [ ] Help team commit to realistic Sprint Goal
- [ ] Confirm Sprint Goal aligns with product vision
- [ ] Ensure selected stories meet Definition of Ready

Daily During Sprint

- [ ] Check Teams for questions (respond within 24h)
- [ ] Review work in progress if requested
- [ ] Note new ideas for backlog (don't add to sprint)
- [ ] Prepare for upcoming refinement session

Mid-Sprint Refinement Checklist

- [ ] Present 5-8 stories for next sprint
- [ ] Explain value/context for each
- [ ] Get team feedback on clarity
- [ ] Adjust stories based on input
- [ ] Ensure stories meet Definition of Ready
- [ ] Facilitate estimation (Planning Poker)

Sprint Review Checklist

- [ ] Prepare demo script (what to show, what value)
- [ ] Test demo beforehand (avoid surprises)
- [ ] Present completed work to stakeholders
- [ ] Explain value delivered
- [ ] Gather feedback
- [ ] Take notes on requested changes
- [ ] Update backlog based on feedback (after meeting)

Sprint Retrospective Checklist

- [ ] Participate actively
- [ ] Listen to team feedback about collaboration
- [ ] Acknowledge areas where you can improve
- [ ] Commit to specific improvements
- [ ] Thank team for honest feedback

Tips for Success

1. Be Decisive

Your job is to make decisions. Don't defer everything to the team or stakeholders.

When asked "Should we do X or Y?"

- ✅ Make a decision based on value
- ❌ Don't say "I don't know, what do you think?" (unless it's a technical question)

2. Be Available

Team needs you. Block time daily for PO work.

Recommended schedule:

- Morning: Check Teams, respond to questions
- Mid-day: Attend Daily Standup (or check-in)
- Afternoon: Backlog work, refinement, stakeholder engagement

3. Be Clear

Vague stories = wasted effort.

Always ask yourself:

- "Can the team build this without guessing?"
- "How will we know it's done?"

- "What value does this deliver?"

4. Prioritize Ruthlessly

You can't build everything.

When prioritizing, ask:

- "What delivers most value soonest?"
- "What do users need most urgently?"
- "What unblocks other work?"

5. Say No

Protecting the team from overload is your job.

Practice saying:

- "That's a great idea, but not for this sprint."
- "We can't do all 10 features well. Let's focus on the top 5."
- "That's valuable, but lower priority than X. Let's revisit in Sprint N+1."

6. Collaborate, Don't Dictate

You need the team's expertise.

Good PO behaviors:

- "Can we achieve this?" (asking)
- "What's realistic this sprint?" (listening)
- "How would you approach this?" (valuing input)

Bad PO behaviors:

- "You must do this." (demanding)
- "I don't care about technical constraints." (dismissing)
- "Just make it work." (abdicating)

Getting Help

When to Ask Staff for Help

- **GDPR questions:** Always consult staff/DPO for privacy requirements
- **Product Vision unclear:** Ask staff to clarify strategic direction
- **Conflicting stakeholder demands:** Escalate for staff to resolve
- **Team conflict about priorities:** Ask Scrum Master and staff for mediation
- **Scope too large for program timeline:** Discuss feasibility with staff

When to Work with Scrum Master

- **Process questions:** "How should refinement sessions work?"
- **Team dynamics:** "Team seems blocked, can you help facilitate?"
- **Ceremony support:** SM helps facilitate, you focus on content
- **Backlog tool questions:** "How do I use Jira effectively?"

When to Collaborate with Team

- **Technical feasibility:** "Is this technically possible in one sprint?"
- **Story splitting:** "How can we break this large story into smaller pieces?"
- **Acceptance criteria:** "Is this clear enough to build and test?"
- **Estimation:** Team estimates effort, not you

Remember

You are learning. Nobody expects perfection in Sprint 1.

Good Product Owners:

- Make decisions (even if not perfect)
- Listen to feedback and adapt
- Communicate clearly
- Prioritize value
- Collaborate with team

You don't need to:

- Know all the answers (ask for help)
- Understand all technical details (focus on WHAT, not HOW)
- Please everyone (make trade-offs)
- Work alone (collaborate)

Your success = Team delivers valuable, working software that solves real problems.

Questions? Ask your Scrum Master, staff coach, or fellow Product Owners.

END OF PRODUCT OWNER ROLE GUIDE